

Approximate Inversion of Discrete Fourier Integral Operators via Hierarchically Semiseparable Matrices

Jingyu Liu

School of Mathematical Sciences, Fudan University

August 31, 2026

Joint work with Prof. Yingzhou Li

1 Introduction

2 Preliminaries

3 Approximate Inversion of FIOs

4 Numerical Results

Fourier integral operator (FIO):

$$u(\mathbf{x}) = \sum_{\boldsymbol{\xi} \in \Xi} a(\mathbf{x}, \boldsymbol{\xi}) e^{2\pi i \phi(\mathbf{x}, \boldsymbol{\xi})} f(\boldsymbol{\xi}), \quad \mathbf{x} \in X.$$

- Space grid $X = \{\mathbf{x}_i = (i_1/n, \dots, i_d/n) : 0 \leq i_1, \dots, i_d < n\}$;
- Frequency grid $\Xi = \{\boldsymbol{\xi}_j = (j_1, \dots, j_d) : -n/2 \leq j_1, \dots, j_d < n/2\}$;
- Amplitude function $a(\mathbf{x}, \boldsymbol{\xi})$: smooth in $(\mathbf{x}, \boldsymbol{\xi})$;
- Phase function $\phi(\mathbf{x}, \boldsymbol{\xi})$: smooth in $(\mathbf{x}, \boldsymbol{\xi})$ for $\boldsymbol{\xi} \neq \mathbf{0}$ and homogeneous of degree 1 in $\boldsymbol{\xi}$, i.e., $\phi(\mathbf{x}, \lambda \boldsymbol{\xi}) = \lambda \phi(\mathbf{x}, \boldsymbol{\xi})$ for any $\lambda > 0$;
- Degree of freedom: $N = n^d$.

Matrix form:

$$\mathbf{u} = \mathbf{K} \mathbf{f}.$$

Free-space wave equation:

$$\begin{aligned}\frac{\partial^2 u}{\partial t^2}(t, \mathbf{x}) &= c^2 \Delta u(t, \mathbf{x}), \\ u(0, \mathbf{x}) &= u_0(\mathbf{x}), \\ \frac{\partial u}{\partial t}(0, \mathbf{x}) &= 0.\end{aligned}$$

The solution is given by

$$u(t, \mathbf{x}) = \frac{1}{2} \left(\int_{\mathbb{R}^d} e^{2\pi i(\mathbf{x} \cdot \boldsymbol{\xi} + c\|\boldsymbol{\xi}\|t)} \widehat{u}_0(\boldsymbol{\xi}) d\boldsymbol{\xi} + \int_{\mathbb{R}^d} e^{2\pi i(\mathbf{x} \cdot \boldsymbol{\xi} - c\|\boldsymbol{\xi}\|t)} \widehat{u}_0(\boldsymbol{\xi}) d\boldsymbol{\xi} \right).$$

$\rightsquigarrow$ The solution operator is the sum of two FIOs with phase functions

$$\phi_{\pm}(\mathbf{x}, \boldsymbol{\xi}) = \mathbf{x} \cdot \boldsymbol{\xi} \pm c\|\boldsymbol{\xi}\|t.$$

Forward computation: Computing $\mathbf{u} = \mathbf{K}\mathbf{f}$ for a given $\mathbf{f}$.

- Wedge partitioning + NUFFT [Candès, Demanet, and Ying 2007];
- Butterfly algorithm [Candès, Demanet, and Ying 2009];
- Butterfly factorization (BF) [Li and Yang 2017; Li, Yang, et al. 2015];
- Tensor butterfly factorization [Kielstra et al. 2025];

Inverse computation: Recovering $\mathbf{f}$ from $\mathbf{u} = \mathbf{K}\mathbf{f}$ for a given $\mathbf{u}$.

- Iterative methods: GMRES / CG + BF;
- Direct methods: BF + $\mathcal{H}$ -matrix + hierarchical interpolative factorization (HIF) [Feliu-Fabá and Ying 2021].

1 Introduction

2 Preliminaries

3 Approximate Inversion of FIOs

4 Numerical Results

FIO matrices admit the [butterfly factorization \(BF\)](#):

$$\mathbf{K} \approx \tilde{\mathbf{K}} = \mathbf{U}^{[L]} \mathbf{B}^{[L]} \dots \mathbf{B}^{[h+1]} \mathbf{M}^{[h]} \mathbf{C}^{[h+1]} \dots \mathbf{C}^{[L]} \mathbf{V}^{[L]},$$

where $L \approx \log N$, $h \approx L/2$ and each factor is a sparse matrix with $\mathcal{O}(N)$ nonzero entries.

- Construction complexity: $\mathcal{O}(N \log N)$.
- Application complexity: $\mathcal{O}(N \log N)$.
- The adjoint $\tilde{\mathbf{K}}^*$ is obtained by applying the adjoint factors in reverse order.

HSS Matrices

Key properties of a **hierarchically semiseparable (HSS)** matrix

- Off-diagonal blocks are **low-rank**;
- Bases are **nested**.

Low-rank off-diagonal blocks

$$D_{\tau} = \begin{bmatrix} D_{\alpha_1} & U_{\alpha_1}^{\text{big}} B_{\alpha_2, \alpha_1}^* U_{\alpha_2}^{\text{big}, *} \\ U_{\alpha_2}^{\text{big}} B_{\alpha_2, \alpha_1} U_{\alpha_1}^{\text{big}, *} & D_{\alpha_2} \end{bmatrix}.$$

Nested basis property

$$U_{\tau}^{\text{big}} = \begin{bmatrix} U_{\alpha_1}^{\text{big}} & \\ & U_{\alpha_2}^{\text{big}} \end{bmatrix} U_{\tau}.$$

In our algorithm

The matrix $\mathbf{G} = \mathbf{K}^* \mathbf{K}$ is approximated by an HSS matrix $\tilde{\mathbf{G}}$.

Illustration of HSS Matrices

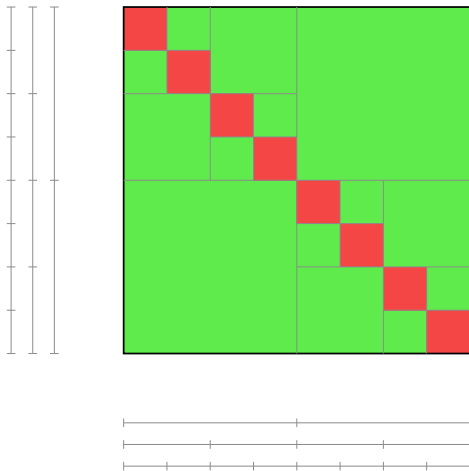
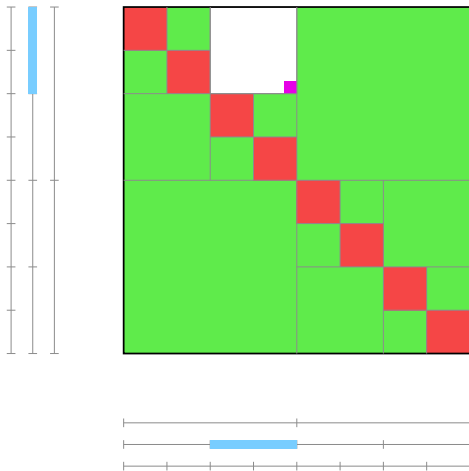


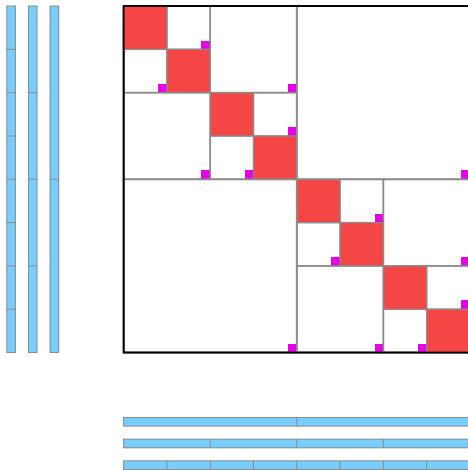
Illustration of HSS Matrices



$$\begin{matrix} \text{Green Block} \end{matrix} = \begin{matrix} \text{Blue Bar} \\ \text{Magenta Square} \\ \text{Blue Bar} \end{matrix}$$

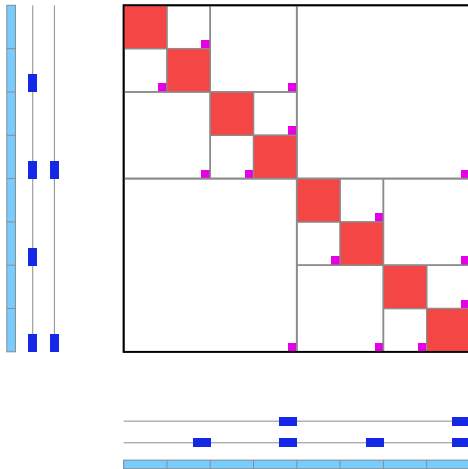
$$H(\mathcal{J}_\tau, \mathcal{J}_\sigma) = U_\tau^{\text{big}} B_{\tau, \sigma} U_\sigma^{\text{big},*}.$$

Illustration of HSS Matrices



$$H(\mathcal{J}_\tau, \mathcal{J}_\sigma) = U_\tau^{\text{big}} B_{\tau, \sigma} U_\sigma^{\text{big},*}.$$

Illustration of HSS Matrices



$$H(\mathcal{J}_\tau, \mathcal{J}_\sigma) = U_\tau^{\text{big}} B_{\tau, \sigma} U_\sigma^{\text{big},*}.$$

$$\begin{bmatrix} \text{blue bar} \end{bmatrix} = \begin{bmatrix} \text{blue bar} & \text{blue bar} \end{bmatrix} \begin{bmatrix} \text{blue block} \end{bmatrix}$$

$$U_\tau^{\text{big}} = \begin{bmatrix} U_{\alpha_1}^{\text{big}} & \\ & U_{\alpha_2}^{\text{big}} \end{bmatrix} U_\tau.$$

ULV Factorization of HSS Matrices

The ULV factorization of an HPD HSS matrix $\mathbf{H}$ takes the form:

$$\mathbf{H} = \mathbf{L}\mathbf{L}^*,$$

where

$$\mathbf{L} = \mathbf{Q}^{[L]} \mathbf{L}^{[L]} \mathbf{\Pi}^{[L]} \dots \mathbf{Q}^{[1]} \mathbf{L}^{[1]} \mathbf{\Pi}^{[1]} \mathbf{L}^{[0]}.$$

- $\mathbf{Q}^{[\ell]}$ is a block diagonal unitary matrix;
- $\mathbf{L}^{[\ell]}$ is a block lower-triangular matrix;
- $\mathbf{\Pi}^{[\ell]}$ is a permutation matrix.

In our algorithm

The ULV factorization of $\tilde{\mathbf{G}}$ is computed in order to efficiently apply $\tilde{\mathbf{F}} = \tilde{\mathbf{G}}^{-1}$.

1 Introduction

2 Preliminaries

3 Approximate Inversion of FIOs

4 Numerical Results

HSS Approximation in FIOs

We approximate $\mathbf{G} = \mathbf{K}^* \mathbf{K}$ by an HSS matrix $\tilde{\mathbf{G}}$.

Suppose that the rank of the HSS blocks at level ℓ is bounded by r_ℓ .

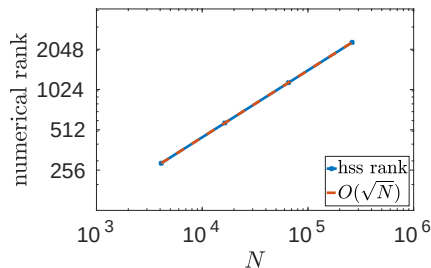
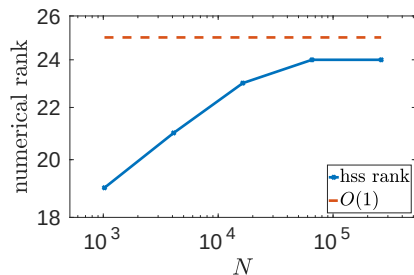
Empirical rank model:

1D

$r_\ell = \mathcal{O}(1)$ independent of the submatrix size.

2D

$r_\ell = \mathcal{O}(n/2^\ell) = \mathcal{O}(\sqrt{\text{submatrix size}})$.



These rank scalings lead to **different** HSS/ULV complexities in 1D and 2D.

Black-box HSS Construction

Goal: construct an HSS approximation of $\mathbf{G} = \mathbf{K}^* \mathbf{K}$ using only matrix-vector products.

Bottom-up process:

- Compute the sampling matrix $\mathbf{Y}^{[\ell]}$ through

$$\mathbf{Y}^{[\ell]} = \mathbf{G}^{[\ell]} \mathbf{\Omega}^{[\ell]} = (\mathbf{U}^{[\ell+1],*} \dots \mathbf{U}^{[L],*}) \mathbf{G} (\mathbf{U}^{[\ell+1]} \dots \mathbf{U}^{[L]}) \mathbf{\Omega}^{[\ell]};$$

- Recover the basis $\mathbf{U}^{[\ell]}$ and the projected diagonal blocks.

Top-down process:

- Assign the matrices $\mathbf{B}^{[\ell]}$ and $\mathbf{D}^{[\ell]}$.

In our algorithm

Matrix-vector products with $\mathbf{G}$ are replaced by fast applications of $\tilde{\mathbf{K}}^* \tilde{\mathbf{K}}$.

Approximate Inversion Algorithm

If K is nonsingular, then

$$K^{-1} = (K^* K)^{-1} K^*.$$

We approximate each component:

- $\tilde{K} \approx K$: BF construction;
- $\tilde{G} \approx G = K^* K$: HSS construction;
- $\tilde{F} = \tilde{G}^{-1} \approx G^{-1}$: HSS ULV factorization.

Therefore,

$$K^{-1} \approx \tilde{F} \tilde{K}^*.$$

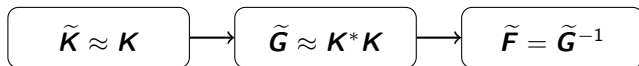
Direct solver:

$$\hat{f} = \tilde{F} \tilde{K}^* u.$$

Preconditioner for CG on the normal equations:

$$\tilde{K}^* \tilde{K} f = \tilde{K}^* u \quad \text{with preconditioner } \tilde{F}.$$

offline stage:



online stage:

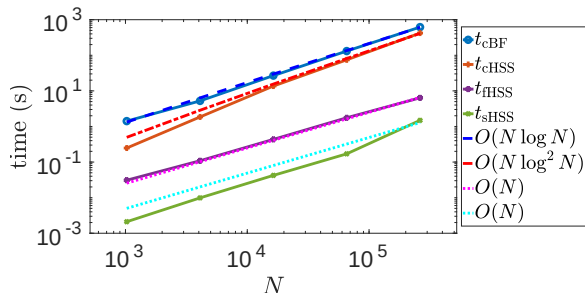
$$K^{-1} \approx \tilde{F} \tilde{K}^*$$

Dimension	construct $\tilde{K}$	apply $\tilde{K}$	construct $\tilde{G}$	factorize $\tilde{G}$	apply $\tilde{F}$
1D	$\mathcal{O}(N \log N)$	$\mathcal{O}(N \log N)$	$\mathcal{O}(N \log^2 N)$	$\mathcal{O}(N)$	$\mathcal{O}(N)$
2D	$\mathcal{O}(N \log N)$	$\mathcal{O}(N \log N)$	$\mathcal{O}(N^{1.5} \log N)$	$\mathcal{O}(N^{1.5})$	$\mathcal{O}(N \log N)$

- 1 Introduction
- 2 Preliminaries
- 3 Approximate Inversion of FIOs
- 4 Numerical Results**

1D FIO With Variable Amplitude: Time Scaling

$$a(\mathbf{x}, \boldsymbol{\xi}) = \sum_{k=1}^{10} \exp\left(-\frac{(\mathbf{x} - \mathbf{x}_k)^2 + ((\boldsymbol{\xi} - \boldsymbol{\xi}_k)/N)^2}{0.1}\right),$$
$$\phi(\mathbf{x}, \boldsymbol{\xi}) = \mathbf{x} \cdot \boldsymbol{\xi} + \rho(\mathbf{x}, \boldsymbol{\xi}), \rho(\mathbf{x}, \boldsymbol{\xi}) = \frac{2 + \sin(2\pi \mathbf{x})}{8} |\boldsymbol{\xi}|.$$



Each component is **consistent with the predicted scaling**.

1D FIO With Variable Amplitude: Iterative Solver

N	t_{iter}	n_{iter}	e_{iter}^1	t_{offline}	t_{piter}	n_{piter}	e_{piter}
2^{10}	5.0e-01	141	5.7e-12	1.7e+00	3.5e-02	4	1.5e-14
2^{12}	2.9e+00	159	5.3e-12	7.2e+00	1.4e-01	4	5.1e-13
2^{14}	1.5e+01	164	6.0e-12	4.1e+01	6.3e-01	4	6.8e-13
2^{16}	7.3e+01	166	5.9e-12	2.1e+02	2.8e+00	4	2.2e-12
2^{18}	3.2e+02	166	6.1e-12	1.0e+03	2.0e+01	5	1.0e-13

- Without preconditioning, CG requires at most 166 iterations;
- With preconditioning, CG converges in only 4-5 iterations;
- About a 20 $\times$ online speedup in iteration time.

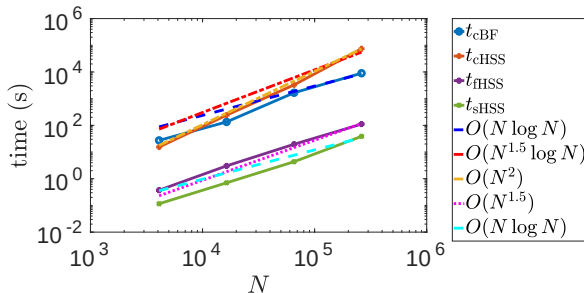
¹Here $\tilde{\mathbf{K}}\mathbf{f}$ is used as a surrogate of the exact $\mathbf{u}$.

2D FIO With Variable Amplitude: Time Scaling

$$a(\mathbf{x}, \boldsymbol{\xi}) = H_0^{(1)}(2\pi\rho(\mathbf{x}, \boldsymbol{\xi}))e^{-2\pi i\rho(\mathbf{x}, \boldsymbol{\xi})},$$

$$\phi(\mathbf{x}, \boldsymbol{\xi}) = \mathbf{x} \cdot \boldsymbol{\xi} + \rho(\mathbf{x}, \boldsymbol{\xi}),$$

$$\rho(\mathbf{x}, \boldsymbol{\xi}) = c(\mathbf{x})\|\boldsymbol{\xi}\|, \quad c(\mathbf{x}) = \frac{3 + \sin(2\pi x_1)\sin(2\pi x_2)}{8}.$$



Each component except the HSS construction is **consistent with the predicted scaling**.

2D FIO With Variable Amplitude: Iterative Solver

N	t_{iter}	n_{iter}	e_{iter}^2	t_{offline}	t_{piter}	n_{piter}	e_{piter}
64^2	1.2e+01	108	1.7e-12	4.3e+01	8.8e-01	6	1.0e-12
128^2	1.4e+02	152	1.8e-12	3.8e+02	6.9e+00	6	1.2e-12
256^2	1.4e+03	214	1.6e-12	5.0e+03	4.9e+01	6	1.0e-12
512^2	1.0e+04	300	1.6e-12	8.3e+04	2.5e+02	6	7.2e-13

- Without preconditioning, the CG iteration count grows to 300;
- With preconditioning, CG converges in only 6 iterations;
- Up to a 40 $\times$ online speedup in iteration time.

²Here $\tilde{K}f$ is used as a surrogate of the exact u .

- **Decomposition:**

$$\mathbf{K}^{-1} = (\mathbf{K}^* \mathbf{K})^{-1} \mathbf{K}^* \approx \tilde{\mathbf{F}} \tilde{\mathbf{K}}^*.$$

- $\tilde{\mathbf{K}} \approx \mathbf{K}$: BF construction;
- $\tilde{\mathbf{G}} \approx \mathbf{G} = \mathbf{K}^* \mathbf{K}$: HSS construction;
- $\tilde{\mathbf{F}} = \tilde{\mathbf{G}}^{-1}$: HSS ULV factorization.

- **Direct solver**

	Offline Complexity	Online Complexity
1D	$\mathcal{O}(N \log^2 N)$	$\mathcal{O}(N \log N)$
2D	$\mathcal{O}(N^{1.5} \log N)$	$\mathcal{O}(N \log N)$

- **Effective preconditioner:**

- 1D: the iteration number drops from the maximum of 166 to 4-5;
- 2D: the iteration number drops from the maximum of 300 to 6.

Thank you!

My homepage:

<https://jingyuliumath.github.io/>

Related paper:

In preparation.

